ESCUELA COLOMBIANA DE INGENIERÍA DESARROLLO ORIENTADO POR OBJETOS PROYECTO INICIAL 2026-2 [:)]

Ciclo No. 1-2-3-4-5/4 — *Refactoring* y Extensión

El proyecto inicial tiene como propósito desarrollar una aplicación que permita simular una situación inspirada en el **Problem I** de la maratón de programación internacional **2025 Slot Machine**. En este simulador los símbolos deben ser de colores diferentes.

CUARTO CICLO

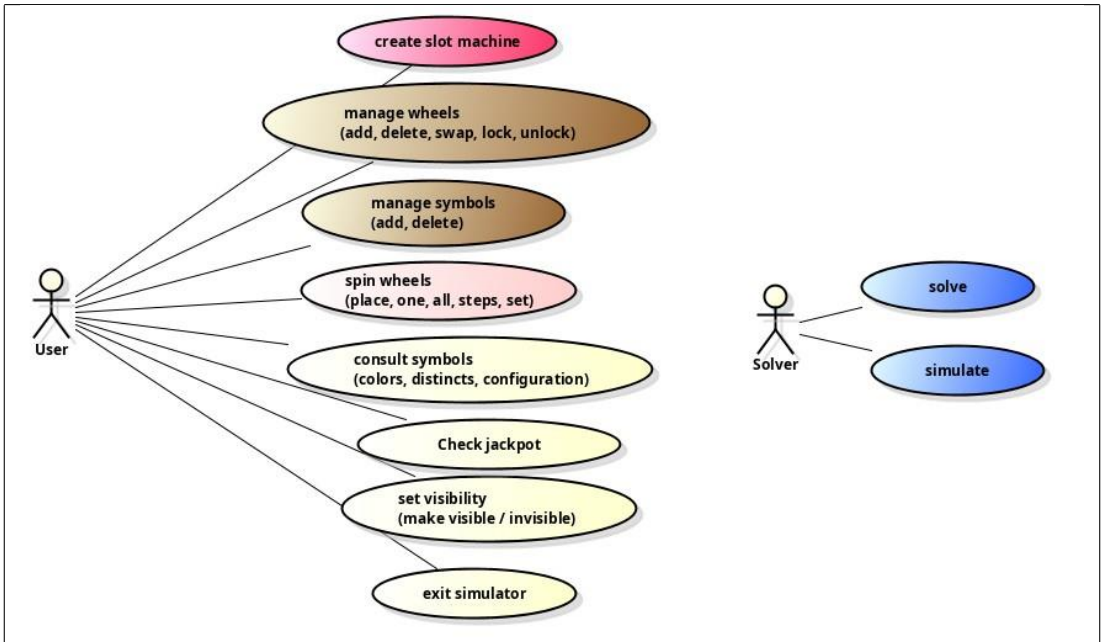
Los requisitos para este ciclo de desarrollo están indicados a continuación. Siempre hay un requisito implícito: el de **EXTENSIBILIDAD**.

MUY IMPORTANTE PLANIFICAR LOS MINI-CICLOS PARA ORIENTAR EL DESARROLLO.

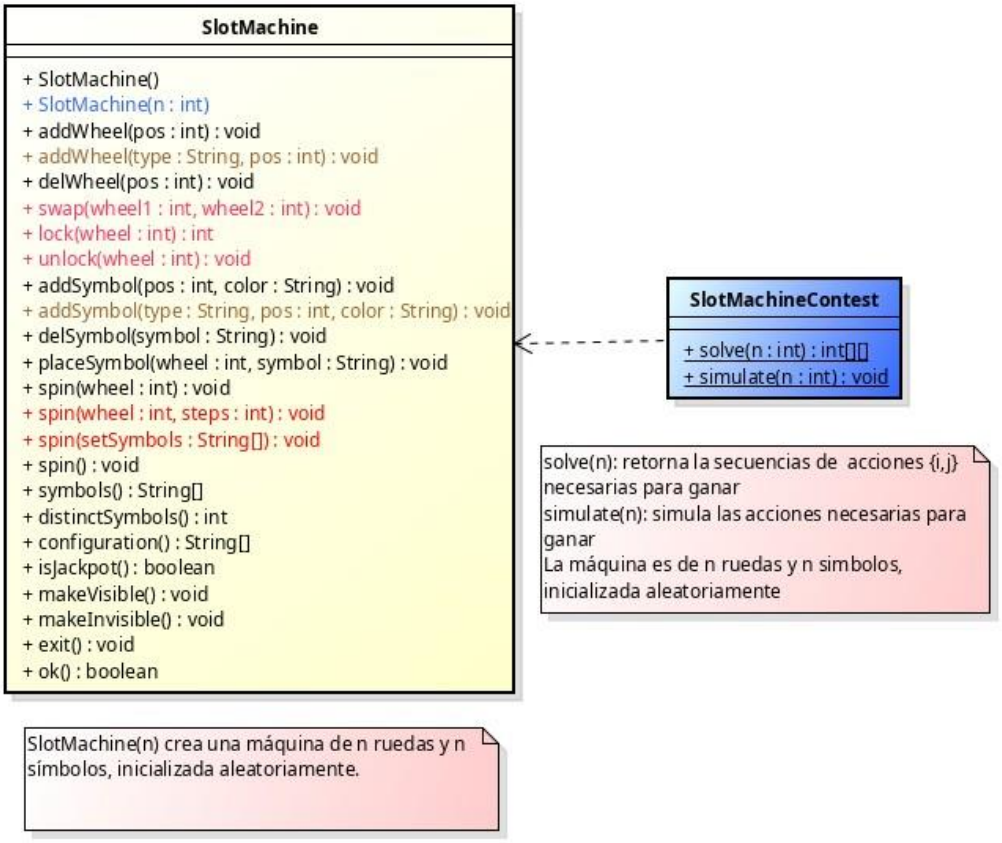
REQUISITOS FUNCIONALES

El simulador debe permitir:

16. Manejar diferentes tipos de ruedas y de símbolos.
17. Ofrecer mínimo tres tipos de ruedas: **normal** (la que tenemos), **lefty**(si hay una rueda a su izquierda, al girar copia su estado), **rebel**(no se deja bloquear, ni intercambiar, ni eliminar).
18. Ofrecer mínimo tres tipos de símbolos: **normal** (el que tenemos), **ephemeral** (en cada giro va decrementando su tamaño hasta quedar como un punto), **shy** (alterna su estado de visible a invisible, cada vez que es seleccionado en la rueda).
19. Ofrecer un nuevo tipo de alguno de los elementos. (Propuesto por ustedes).



REQUISITOS DE DISEÑO



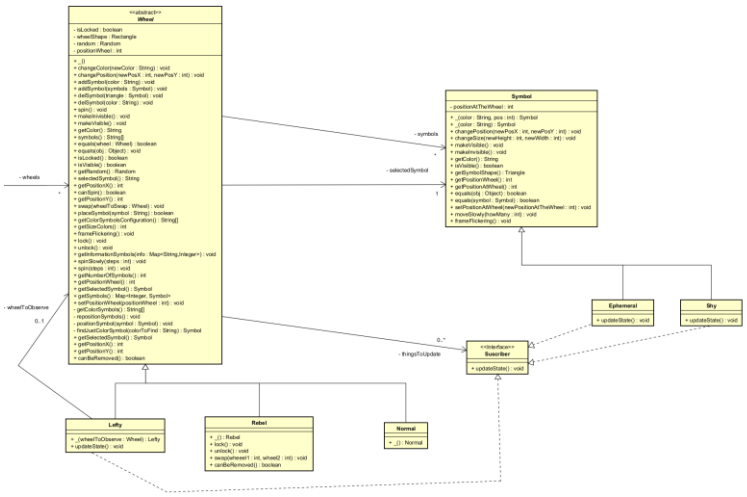
REQUISITOS DE USABILIDAD

1. Los elementos de diferentes tipos deben poder distinguirse claramente visualmente.

REQUISITOS DE ENTREGA

Los productos de esta entrega son:

1. Diseño completo en la herramienta astah



2. Implementación siguiendo los estándares de codificación y documentación de java.
3. Casos de pruebas de unidad de su proyecto: SlotMachineC4Test
4. Mínimo dos casos de prueba de unidad compartidos en la clase SlotMachineCC4Test
5. Dos pruebas de aceptación preparadas para la presentación.
6. Documento de retrospectiva del proyecto. (Ver ciclo uno)
   Es necesario incluir la retrospectiva de este ciclo y de los anteriores.
   DEFINIR EL ESTADO EN TERMINOS DE LOS MINI-CILOS PLANIFICADOS

#Pregunta	Pregunta	Ciclo1	Ciclo 2	Ciclo 3
1.	¿Cuáles fueron los mini-ciclos definidos? Justifíquenlos.	Los mini-ciclos definidos se pueden evidenciar en las partes siguientes de este documento.	Los mini-ciclos definidos se pueden evidenciar en las partes siguientes de este documento.	Los mini-ciclos defin este documento.
2.	¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿por qué?	Completo.	Completo.	Completo.
3.	¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)	15 horas cada uno.	18 horas cada uno.	18 horas cada uno.
4.	¿Cuál consideran fue el mayor logro? ¿Por qué?	Aplicar la práctica XP llamada Pair Programming.	Aplicar la práctica TDD porque está al aplicarse brinda cierta seguridad en que las funcionalidades que se exigen en el proyecto se cumplan.	El mayor logro fue e
5.	¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?	Utilizar GitHub, esto se fue aprendiendo a medida que se utilizaba para guardar el proyecto.	Entender los requerimientos que se pedían, lo que se hizo fue hablar con la profesora para asegurar lo que se creía de cada requerimiento.	El mayor logro fue h pruebas a mano co
6.	¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?	Comunicarnos.	Lo que pudimos hacer fue redistribuirnos trabajo y explicarnos de manera sencilla lo que hizo cada uno.	N-N

7.	Considerando las prácticas XP incluidas en los laboratorios. ¿cuál fue la más útil? ¿por qué?	Simplicidad, Facilito el proceso de programar solo lo esencial y después poderlo extender.	La comunicación con el cliente o profesor fue muy necesaria para cumplir los requisitos y esta es una de las prácticas XP, estar en constante diálogo con el cliente.	Simplicidad, Facilito el proceso d
8.	¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.	**Referencias:** Google. (2026). *Gemini* (Geminii Flash 3.6 y Pro) [Modelo de lenguaje de gran tamaño]. https://gemini.google.com Anthropic. (2026). *Claude* (Versión [indicar versión, ej. Claude 3.6 Sonnet]) [Modelo de lenguaje de gran tamaño]. https://claude.ai	**Referencias:** Martin, R. C. (2012). *Código limpio: Manual de estilo para el desarrollo ágil de software* (2.ª ed.). Anaya Multimedia. Google. (2026). *Gemini* (Geminii Flash 3.6 y Pro) [Modelo de lenguaje de gran tamaño]. https://gemini.google.com Anthropic. (2026). *Claude* (Versión [indicar versión, ej. Claude 3.6 Sonnet]) [Modelo de lenguaje de gran tamaño]. https://claude.ai	**Referencias:** • Escuela Co *de entrega* • https://open

Desarrollo del ciclo 1:

#Miniciclo	SlotMachine	Wheel	Symbol
1	☑ ~~SlotMachine()~~ ☑ ~~addWheel(int pos)~~ ☑ ~~delWheel(int pos)~~	☑ ~~Wheel()~~ ☑ ~~changColor(color: String)~~ ☑ ~~changePositionX(pos: int)~~	☑ ~~Symbol(color: string)~~ ☑ ~~changePositionx(pos: int)~~ ☑ ~~changePositiony(pos: int)~~

	☑ ~~addSymbol(int pos, String color)~~ ☑ ~~delSymbol(String symbol)~~	☑ ~~changePositionY(pos: int)~~	
2	☑ ~~symbols()~~ ☑ ~~placeSymbol(int wheel, String symbol)~~ ☑ ~~distinctSymbols()~~ ☑ ~~configuration()~~	☑ ~~addSymbol(Symbol: String)~~ ☑ ~~delSymbol(Symbol: String)~~ ☑ ~~selectSymbol()~~	☑ ~~makeVisible()~~ ☑ ~~makeInvisible()~~
3	☑ ~~spin(int wheel)~~ ☑ ~~spin()~~ ☑ ~~isJackpot()~~ ☑ ~~configuration()~~ ☑ ~~ok()~~ ☑ ~~exit()~~	☑ ~~spin()~~ ☑ ~~symbols()~~	
4	☑ ~~makeVisible()~~ ☑ ~~makeInvisible()~~	☑ ~~makeVisible()~~ ☑ ~~makeInvisible()~~	

Desarrollo del ciclo 2:

#Miniciclo	SlotMachine	Wheel	Symbol
1	☑ ~~swap(wheel1 : int, wheel2 : int)~~ ☑ ~~lock(wheel : int)~~ ☑ ~~unlock(wheel : int)~~	☑ ~~agregar atributo isLocked y~~ ☑ ~~agregar lógica de spin~~	
2	☑ ~~spin(wheel : int, steps : int)~~ ☑ ~~spin(setSymbols : String[])~~	☑ ~~Spin(steps:int)~~ ☑ ~~spinSlowly(steps:int)~~	

Desarrollo del ciclo 3:

#Miniciclo	SlotMachineContest	Pruebas	
1	~~printOI(int i, int u, SlotMachine machine)~~ ~~different(SlotMachine machine)~~	~~Establecer prueba de depuracion~~	
2	~~identical(SlotMachine machine)~~ ~~simulate(int n)~~ ~~solve(int n)~~		

Desarrollo del ciclo 4:

#Miniciclo	SlotMachine	Wheel e hijas	Symbol e hijas
1		~~public Lefty()~~ ~~Public Rebel()~~	~~public Shy(String color)~~ ~~Ephemeral(String color)~~ ~~Rotation(String color)~~
2		~~canSpin()~~ ~~spin(Wheel leftWheel)~~ ~~lock()~~ ~~unlock()~~ ~~swap(Wheel wheelToSwap)~~ ~~validateDeletion()~~	~~efect()~~ ~~reduce()~~ ~~rotating()~~

3	~~addWheel(String type, int pos)~~ ~~spin(int wheel)~~		
4	~~spin(int wheel, int steps)~~ ~~spin(String[] setSymbols)~~		

Los productos los deben publicar en un repositorio Git y deben publicar en Moodle un archivo txt que contenga la dirección URL. El nombre del archivo .txt debe ser la concatenación de los primeros apellidos más la primera letra del segundo apellido de los autores, ordenados alfabéticamente. Publicar productos a revisión: Semana 10 - 3 octubre

ESCUELA COLOMBIANA DE INGENIERÍA

DESARROLLO ORIENTADO POR OBJETOS

PROYECTO INICIAL 2026-2

Ciclo No. 5/4 — *Refactoring* y Extensión

El proyecto inicial tiene como propósito desarrollar una aplicación que permita simular una situación inspirada en el **Problem I** de la maratón de programación internacional **2025 Slot Machine**. En este simulador los símbolos deben ser de colores diferentes.

CIERRE

El objetivo de esta entrega es evaluar y perfeccionar la calidad del simulador mediante el uso de herramientas modernas de desarrollo.

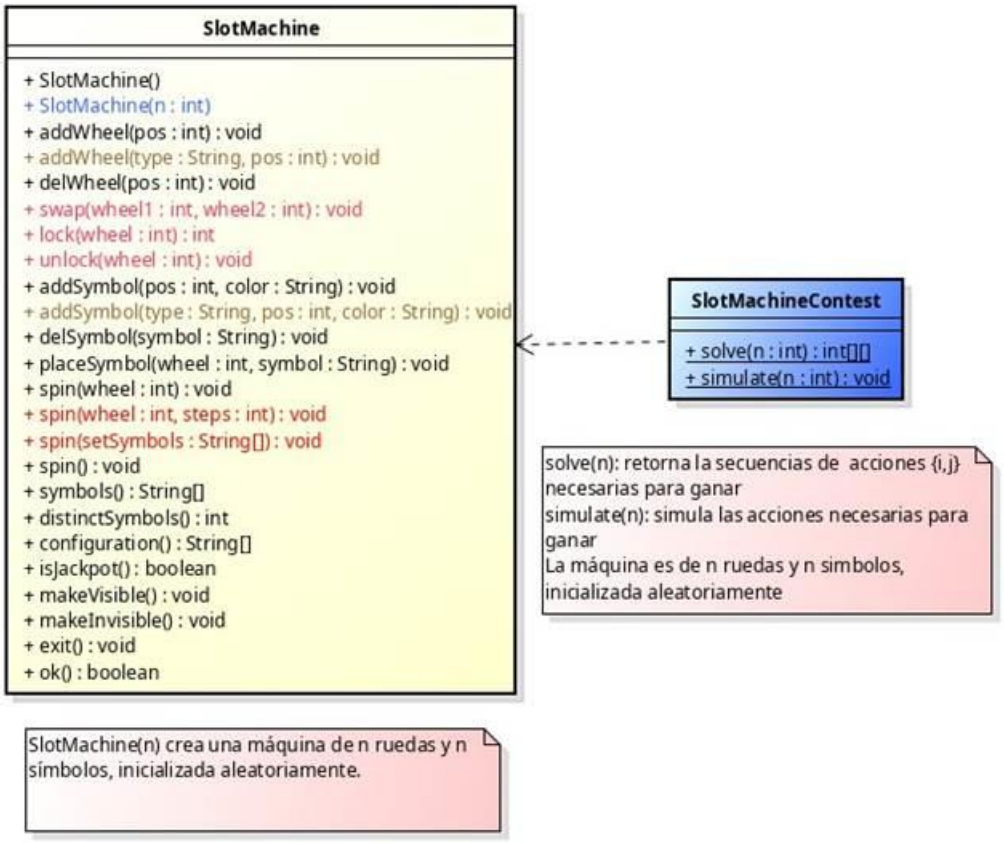
REQUISITOS FUNCIONALES

Para eso se espera que en esta entrega se tenga:

1. El proyecto en IDE VS Code
2. Informe de análisis dinámico: informe de cubrimiento de pruebas de unidad generado con JaCoCo.

3. Informe de análisis estático: informe de cumplimiento de reglas de calidad para java generado con PMD.

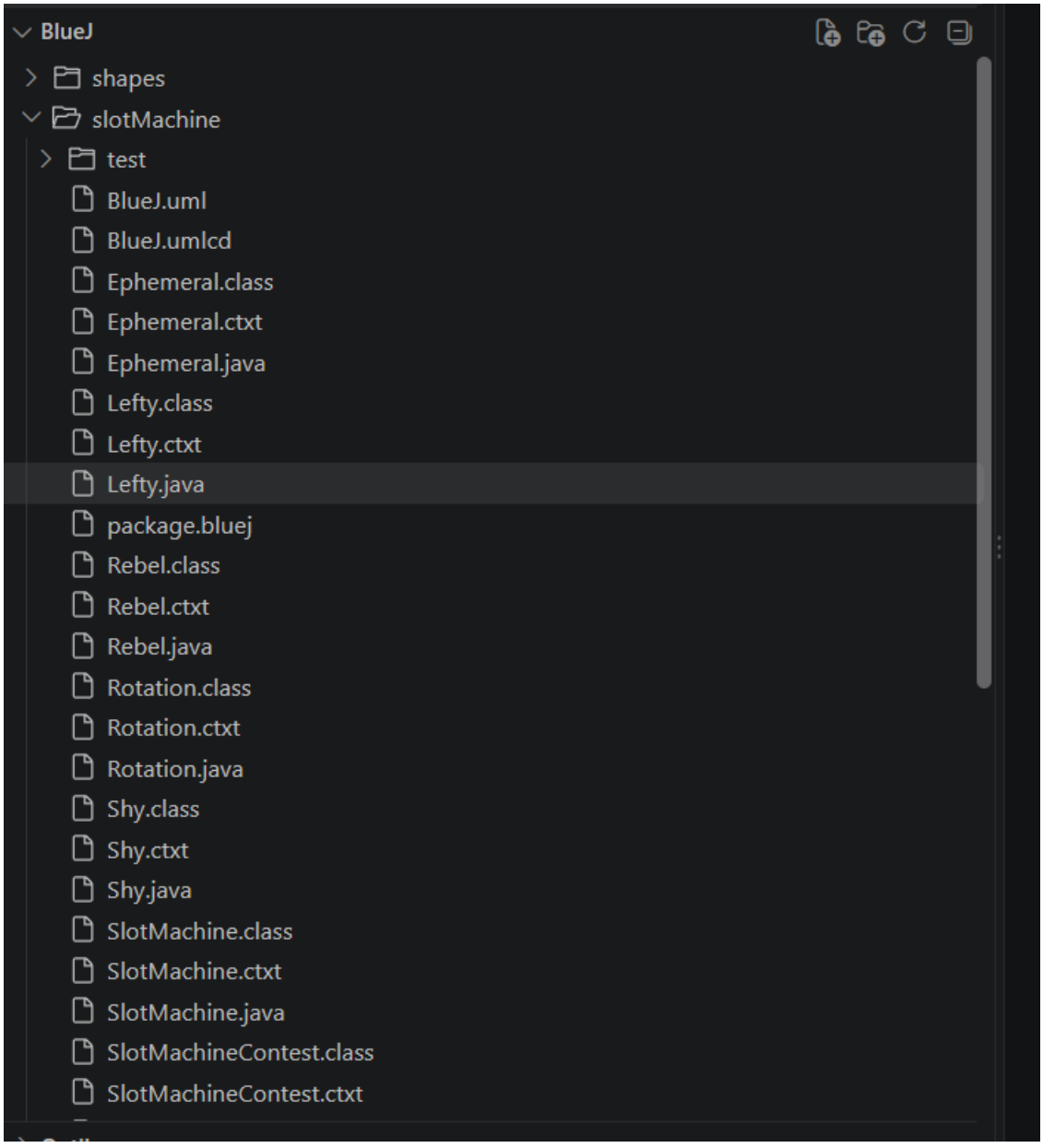
REQUISITOS DE DISEÑO


SlotMachine
+ SlotMachine()
+ SlotMachine(n : int)
+ addWheel(pos : int) : void
+ addWheel(type : String, pos : int) : void
+ delWheel(pos : int) : void
+ swap(wheel1 : int, wheel2 : int) : void
+ lock(wheel : int) : int
+ unlock(wheel : int) : void
+ addSymbol(pos : int, color : String) : void
+ addSymbol(type : String, pos : int, color : String) : void
+ delSymbol(symbol : String) : void
+ placeSymbol(wheel : int, symbol : String) : void
+ spin(wheel : int) : void
+ spin(wheel : int, steps : int) : void
+ spin(setSymbols : String[]) : void
+ spin() : void
+ symbols() : String[]
+ distinctSymbols() : int
+ configuration() : String[]
+ isJackpot() : boolean
+ makeVisible() : void
+ makeInvisible() : void
+ exit() : void
+ ok() : boolean
SlotMachineContest
+ solve(n : int) : int[][]
+ simulate(n : int) : void
solve(n): retorna la secuencias de acciones {i,j} necesarias para ganar
simulate(n): simula las acciones necesarias para ganar
La máquina es de n ruedas y n simbolos, inicializada aleatoriamente
SlotMachine(n) crea una máquina de n ruedas y n símbolos, inicializada aleatoriamente.

ENTREGA

Los productos de esta entrega son:

1. El proyecto en IDE VS Code
   - Instalando la extensión de Java para VS y abriendo el proyecto nmuestra lo siguiente:



   - Se evidencia los archivos de blueJ y cada clase test tiene problemas de importación:

```
package slotMachine;

import org.junit.Test;    The import org.junit cannot be resolved
import static org.junit.Assert.*;    The import org.junit cannot be resolved
import slotMachine.Ephemeral;

public class SymbolEphemeralTest {
```

2. Informe de análisis dinámico: informe de cubrimiento de pruebas de unidad generado con JaCoCo.
   Despues de modificar el codigo para que funcione en VS y instalar JaCoco, se crea un archivo: pom.xml
   El xml que se coloca es el sugiente:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">
 <modelVersion>4.0.0</modelVersion>

 <groupId>edu.eci</groupId>
 <artifactId>headsoccer</artifactId>
 <version>1.0-SNAPSHOT</version>

 <properties>
  <maven.compiler.source>1.8</maven.compiler.source>
  <maven.compiler.target>1.8</maven.compiler.target>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
 </properties>

 <dependencies>
  <dependency>
   <groupId>junit</groupId>
   <artifactId>junit</artifactId>
   <version>4.13.2</version>
  </dependency>
 </dependencies>

 <build>
  <sourceDirectory>${project.basedir}</sourceDirectory>
  <testSourceDirectory>${project.basedir}</testSourceDirectory>

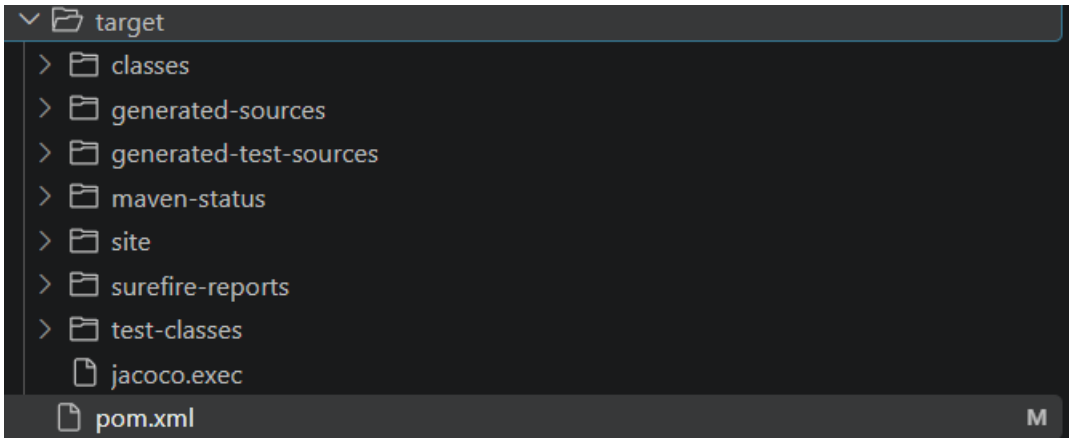
  <plugins>
   <plugin>
    <groupId>org.jacoco</groupId>
    <artifactId>jacoco-maven-plugin</artifactId>
    <version>0.8.10</version>
    <executions>
     <execution>
      <goals>
       <goal>prepare-agent</goal>
      </goals>
     </execution>
     <execution>
      <id>report</id>
      <phase>test</phase>
      <goals>
```

```
            <goal>report</goal>
          </goals>
        </execution>
      </executions>
    </plugin>
  </plugins>
 </build>
</project>
```

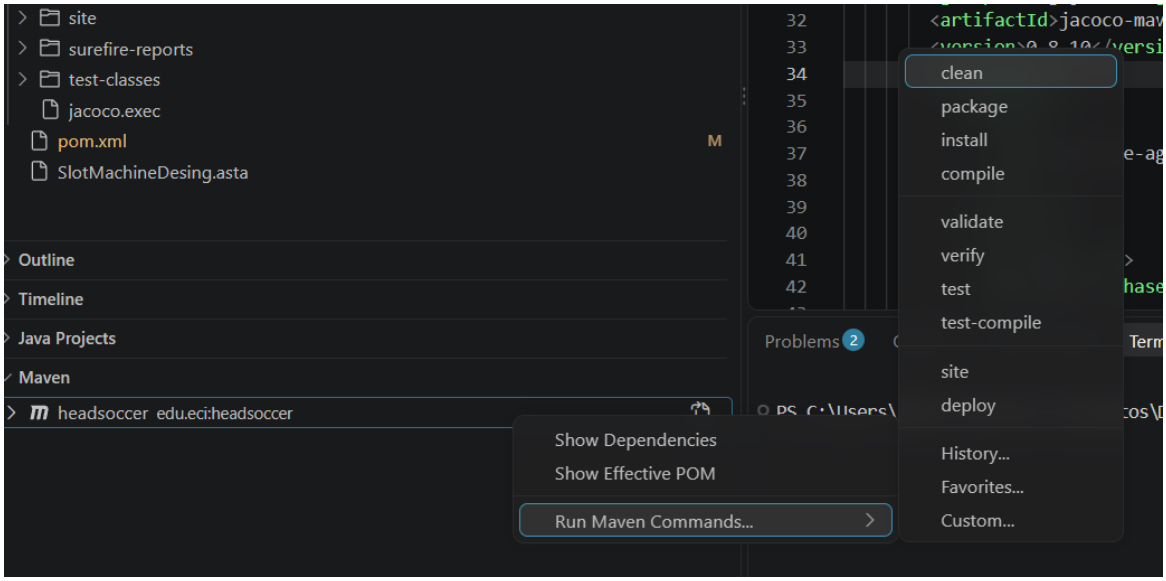
Con ayuda de géminis, explicare cada parte del código y que hace:

"Encabezado XML (<?xml...>, <project>, <modelVersion>): Son etiquetas obligatorias del sistema. Simplemente le indican al computador que este es un archivo de configuración válido para Maven.
Datos del Proyecto (<groupId>, <artifactId>, <version>): Es la identificación única del proyecto. Indica que la organización es edu.eci, el programa se llama headsoccer y actualmente está en una versión de desarrollo (1.0-SNAPSHOT).
Propiedades (<properties>): Son las reglas básicas del entorno. Aquí se configura que el código está escrito para Java 8 (1.8) y que debe usar el formato de texto UTF-8 para evitar errores con caracteres como tildes o la letra ñ.
Dependencias (<dependencies>): Es la lista de librerías externas que el proyecto necesita descargar para funcionar. En este bloque se está incluyendo JUnit, la herramienta necesaria para programar y ejecutar pruebas unitarias.
Carpetas de Construcción (<sourceDirectory>, <testSourceDirectory>): Por defecto, Maven busca el código en unas subcarpetas muy específicas. Estas etiquetas cambian esa regla y le dicen que tanto el código principal como las pruebas están guardados directamente en la carpeta raíz del proyecto.
Plugins (<plugins>): Son programas extra que se ejecutan durante la construcción. Aquí se configura JaCoCo, una herramienta que se ejecuta junto con las pruebas de JUnit y genera un reporte para mostrar qué porcentaje de nuestro código fue probado exitosamente (cobertura de código)."
— Explicación generada por Gemini.

- Después se crea automaticamente la siguiente carpeta:



- Para generar el informe con Maven en la terminal: mvn clean test o tambien se puede en:



- Para ver el informe entramos al index.html:

headsoccer

headsoccer

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
slotMachine		70%		48%	155	304	205	725	28	126	1	12
default		0%		0%	22	22	58	58	6	6	1	1
shapes		85%		70%	21	63	57	274	16	51	0	7
slotMachine.test		99%		100%	2	76	4	421	2	52	0	6
Total	1,251 of 6,157	79%	223 of 460	51%	200	465	324	1,478	52	235	2	26

Explicacion:
(géminis lo reescribió para que tenga una estructura formal y una explicación más clara):

- Element: Indica el nombre del paquete de código o módulo que está siendo analizado.
- Missed Instructions y Cov: Muestran de forma visual mediante barras y porcentualmente la cantidad de código base que fue probado frente al que no fue probado.
- Missed Branches y Cov: Exponen gráficamente y en porcentaje la evaluación de los caminos lógicos, como las estructuras condicionales, verificando si se probaron todas sus salidas posibles.
- Missed y Cxty: Detallan la cantidad de escenarios de complejidad que faltaron por validar frente a la complejidad total del paquete.
- Missed y Lines: Contabilizan el número exacto de líneas de código no evaluadas en contraste con el total de líneas programadas.
- Missed y Methods: Cuantifican los métodos o funciones que no fueron ejecutados por las pruebas sobre el total de métodos creados.
- Missed y Classes: Señalan la cantidad de clases que carecen de pruebas frente al total de clases existentes en el paquete.
- Fila slotMachine: Presenta una cobertura de instrucciones del 70% y una cobertura de ramas del 48%, revelando que 205 líneas de código quedaron sin probar de un total de 725.
- Fila default: Refleja un 0% de cobertura general, lo que indica un déficit total de validación, dejando sus 58 líneas y su única clase sin ninguna prueba asociada.
- Fila shapes: Muestra un desempeño favorable con un 85% de cobertura en instrucciones y un 70% en ramas, reportando 57 líneas sin probar de las 274 totales.

- Fila slotMachine punto test: Destaca como el componente con mayor validación, alcanzando un 99% en instrucciones y un 100% en ramas, dejando únicamente 4 líneas huérfanas de 421.

Fila Total: Resume el estado global de todo el proyecto, consolidando un 79% de cobertura de instrucciones y un 51% en ramas, con 324 líneas faltantes por revisar de las 1,478 totales del sistema.

3. Informe de análisis estático: informe de cumplimiento de reglas de calidad para java generado con PMD.

- Se agrega a pom.xml lo siguente:

```
<plugin>
 <groupId>org.apache.maven.plugins</groupId>
 <artifactId>maven-pmd-plugin</artifactId>
 <version>3.21.2</version>
 <configuration>
  <linkXRef>false</linkXRef>
  <rulesets>
   <ruleset>/category/java/bestpractices.xml</ruleset>
   <ruleset>/category/java/codestyle.xml</ruleset>
   <ruleset>/category/java/errorprone.xml</ruleset>
  </rulesets>
 </configuration>
</plugin>
```

- Abrimos pmd.html:

```
PS C:\Users\pablo\OneDrive\Documentos\DOPO\Proyecto1-DOPO\Ciclo5> mvn pmd:pmd
[INFO] Scanning for projects...
[INFO] Loaded 24179 auto-discovered prefixes for remote repository central (prefixes-central.txt)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-pmd-plugin/3.21.2/maven-pmd-plugin-3.21.2.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-pmd-plugin/3.21.2/maven-pmd-plugin-3.21.2.pom (16
kB at 49 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/40/maven-plugins-40.pom
```

Last Published: 2026-10-21 | Version: 1.0-SNAPSHOT

Built by maven

PMD Results

The following document contains the results of PMD 6.55.0.

Violations By Priority

Priority 1

ProblemSlotMachine/slotmachine.java

Rule	Violation	Line
ClassNamingConventions	The class name 'slotmachine' doesn't match '[A-Z][a-zA-Z0-9]*'	9-103

Proyecto/slotMachine/SlotMachine.java

Rule	Violation	Line
ConstructorCallsOverridableMethod	Overridable method 'addWheel' called during object construction	48
ConstructorCallsOverridableMethod	Overridable method 'addSymbol' called during object construction	51
ConstructorCallsOverridableMethod	Overridable method 'spin' called during object construction	54

Proyecto/slotMachine/test/SlotMachineAceptanceTest.java

Rule	Violation	Line
VariableNamingConventions	Only variables that are final should contain underscores (except for underscores in standard prefix/suffix), 'NUMBER_WHEELS_TO_ADD' is not final.	30
LocalVariableNamingConventions	The local variable name 'NUMBER_WHEELS_TO_ADD' doesn't match '[a-z][a-zA-Z0-9]*'	30
VariableNamingConventions	Variables should start with a lowercase character, 'NUMBER_WHEELS_TO_ADD' starts with uppercase character.	30
VariableNamingConventions	Only variables that are final should contain underscores (except for underscores in standard prefix/suffix), 'NUMBER_WHEELS_TO_ADD' is not final.	40

Explicación generada por geminis, en el punto posterior se ve la corrección de los errores:

- "**Prioridad 1 (Riesgos Críticos y Convenciones Estrictas):**

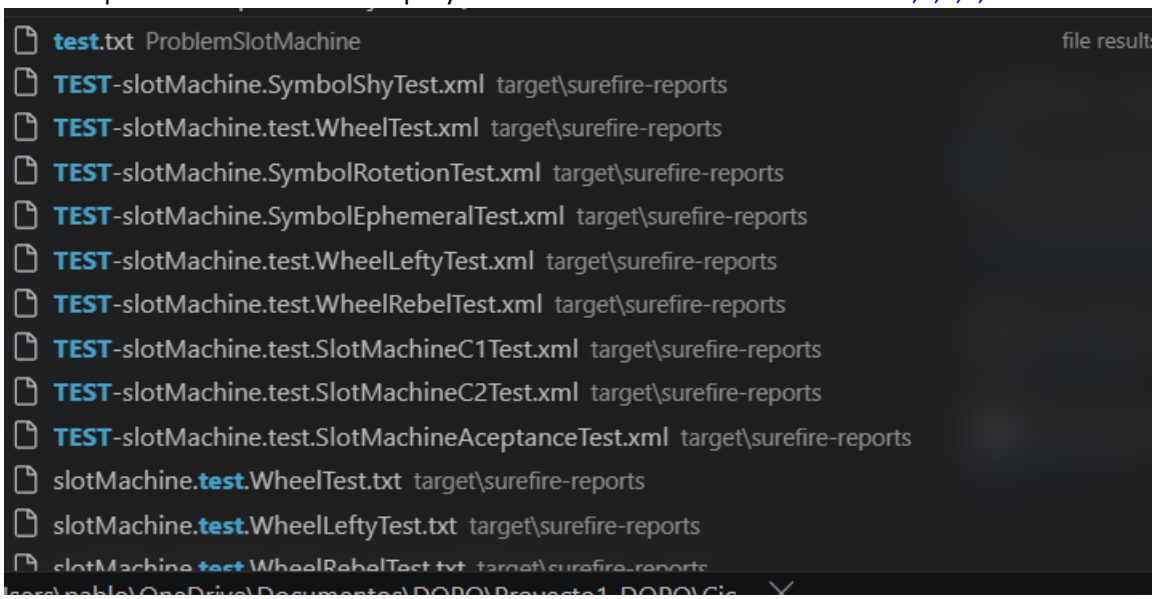
- En `Proyecto/slotMachine/SlotMachine.java`, el constructor hace llamadas directas a los métodos sobreescribibles `addWheel`, `addSymbol` y `spin` durante la construcción del objeto, lo cual es una falla estructural.
- En `ProblemSlotMachine/slotmachine.java`, el nombre de la clase incumple la convención de iniciar con mayúscula (`[A-Z][a-zA-Z0-9]*`).
- Las clases de prueba (`SlotMachineAceptanceTest.java`, `SlotMachineC1Test.java` y `SlotMachineC2Test.java`) presentan múltiples errores de convención por variables locales como `NUMBER_WHEELS_TO_ADD` y `NUMBER_WHEELS_TO_PLACE`. Estas variables no están declaradas como `final`, por lo que no deberían usar mayúsculas ni guiones bajos.
- **Prioridad 2 (Malas Prácticas e Interfaz):**
- Las clases `Proyecto/slotMachine/Symbol.java` y `Proyecto/slotMachine/Wheel.java` incluyen métodos cuyos nombres y cantidad de parámetros son sospechosamente similares al método estándar `equals(Object)`, lo que sugiere un error de sobrescritura.
- Hay advertencias por reasignar valores a los parámetros recibidos, específicamente `distance` (en `Figure.java`) y `pos` o `wheel` (en `SlotMachine.java`).
- Se advierte el uso inapropiado de impresiones directas en consola mediante `System.out.println` en `SlotMachine.java`.
- **Prioridad 3, 4 y 5 (Mantenibilidad, Estilo y Flujo de Datos):**
- **Estilo y Sintaxis:** Múltiples clases carecen de paquetes nombrados, de llaves en los bloques `if` (ControlStatementBraces), o utilizan literales mágicos dentro de las condiciones. Se reportan advertencias por el uso de nombres de variables demasiado cortos (`x`, `y`, `n`, `k`) o excesivamente largos.
- **Acoplamiento y Diseño:** Se señala la dependencia de implementaciones concretas como `HashMap` y `ArrayList` en lugar de usar sus respectivas interfaces, e importaciones no utilizadas o duplicadas. Gran parte del código omite el modificador `final` en parámetros locales que no cambian de valor.
- **Anomalías de Flujo:** Existen numerosos reportes "DD" (Variable redefinida sin uso previo) y "DU" (Variable definida pero posiblemente indefinida al momento de usarse) en clases clave como `slotmachine.java`, `SlotMachineContest.java`, y `Wheel.java`."
  — Explicación generada por Gemini.

4. Diseño completo en la herramienta astah

Completado, se guardó en ciclo 5 y una copia en el directorio principal.

5. Implementación siguiendo los estándares de codificación y documentación de java en VS Code

6. Casos de pruebas de unidad de su proyecto en VS Code : SlotMachine*Test: 1,2,3,4,Contest



7. Una prueba de aceptación ejecutando en VS Code.

Por indicaciones de la profesora, no se codifica, el enunciado es:

Primero, acomodé la máquina para que mostrara los colores que yo quería: magenta, verde y azul, y comprobé que quedaran exactamente en ese orden. Luego, cambié de posición la primera y la tercera rueda, y al hacer eso, vi que los colores ahora estaban al revés: azul, verde y magenta. Después de esto, bloqueé la primera y la tercera rueda para probar qué pasaba. Cuando hice esto e intenté darle un giro completo a la máquina, me di cuenta de que solo dio vueltas la rueda del medio; las que había bloqueado ni se movieron, y me saltó una ventanita de error en la pantalla. Para estar seguro de que de verdad estaban trabadas, intenté cambiarlas de lugar de nuevo y hasta traté de eliminar la tercera rueda, pero el programa no me dejó hacer absolutamente nada. Las ruedas se quedaron ahí quietas, intactas, y me volvió a salir el mensaje de que no estaba permitido hacer eso.

8. Informe de análisis dinámico; resultado inicial y resultado final.
   Descripción de las decisiones tomadas después de estudiar el resultado inicial. Meta: más de 75 % de cubrimiento de código.

   Ya cumplia desde el inicio del ciclo 5

9. Informe de análisis estático: resultado inicial y resultado final.
   Descripción de las decisiones tomadas después de estudiar el resultado inicial. Meta: cumplimiento de todas las reglas de prioridad alta (rojo).
   Correcciones que se hicieron (texto de lo que se debia correguir generado por geminis) :
   - Prioridad 1 (Errores Críticos de Diseño y Convención)
     1. Renombra la clase slotmachine para que cumpla el estándar [A-Z][a-zA-Z0-9]* (por ejemplo, SlotMachine).
     2. Evita llamar a métodos que puedan ser sobrescritos (addWheel, addSymbol, spin) dentro del constructor en SlotMachine.java.
     3. Cambia el nombre de variables locales como NUMBER_WHEELS_TO_ADD a formato camelCase; las mayúsculas con guiones bajos están reservadas estrictamente para variables final.

- Prioridad 2 (Malas Prácticas y Prevención de Bugs)
    1. Evita modificar o reasignar parámetros recibidos en métodos (como distance, pos o wheel).
    2. Elimina el uso de System.out.println en el código de producción y utiliza un sistema de logs apropiado.
    3. Renombra o cambia la firma de los métodos en Symbol.java y Wheel.java que son sospechosamente parecidos a equals(Object) para no interferir con el método estándar de Java.
- Prioridad 3 (Estilo, Legibilidad y Estructura)
    1. Declara un paquete para las clases que no lo tienen y asegúrate de que todos los nombres de paquete usen exclusivamente minúsculas.
    2. Añade la palabra reservada final a parámetros y variables locales cuyo valor no cambia tras la asignación.
    3. Utiliza siempre llaves {} para los bloques de sentencias condicionales (if) y reestructura tus métodos para que tengan un único punto de retorno (return) al final.
    4. Reduce el acoplamiento utilizando interfaces genéricas (Map, List) en lugar de instanciar contra implementaciones concretas como HashMap o ArrayList.
    5. Modifica las pruebas de JUnit incluyendo mensajes descriptivos en los constructores de los asserts y limitando la cantidad de aserciones por prueba unitaria.
    6. Reemplaza números "mágicos" (literales numéricos sin contexto) en los bloques condicionales por variables o constantes representativas.
    7. Elimina los bloques catch vacíos, retira llamadas explícitas a System.exit(), y cambia variables con nombres de una sola letra (n, x, y) por nombres más explicativos.
- Prioridad 4 y 5 (Limpieza Superficial y Análisis de Flujo)
    1. Borra todas las importaciones no usadas (como java.util.Arrays o comodines slotMachine.*) y evita importaciones duplicadas.
    2. Separa las declaraciones múltiples dejando una sola declaración por línea.
    3. Repara anomalías de flujo (marcos DU y DD) generadas al asignar valores a variables que nunca son utilizadas posteriormente o al sobreescribir valores antes de leerlos.

— texto generado por Gemini.

Se solucionarion todos los errores de prioridad 1 que no interfería con el codigo ni con la lógica planteada.

Violations By Priority

Priority 1

ProblemSlotMachine/slotmachine.java

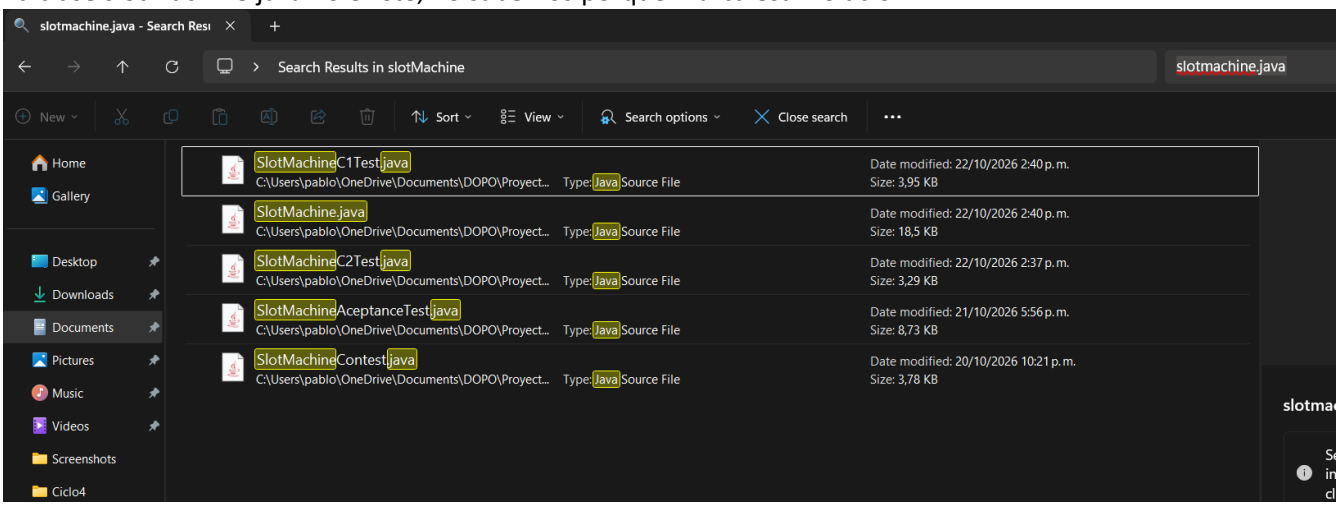
Rule	Violation	Line
ClassNamingConventions	The class name 'slotmachine' doesn't match '[A-Z][a-zA-Z0-9]*'	9–103

Proyecto/slotMachine/SlotMachine.java

Rule	Violation	Line
ConstructorCallsOverridableMethod	Overridable method 'addWheel' called during object construction (call stack: [addSlotMachine, addWheel])	44

Priority 2

La clase slotmachine.java no existe, no sabemos porque marca esa violación.



10. Documento de retrospectiva del proyecto. (Ver ciclo uno)
    Los miniciclos de los anteriores ciclos ya estan definidos anteriormente en el documento.

Los productos los deben publicar en Moodle un archivo zip que contenga todos los productos. El nombre del archivo .zip debe ser la concatenación de los primeros apellidos más la primera letra del segundo apellido de los autores, ordenados alfabéticamente.

Publicar productos a revisión: Semana 12 - 24 octubre